

Reference Notes

Week 5: String Immutability and Memory

7th Grade Computer Science - Module 2

1 Introduction

1.1 This Week's Big Question

Why do some data types behave differently when we try to change them? This week, we'll uncover one of Python's most important secrets: strings are immutable - they cannot be changed! This might sound strange at first (didn't we "change" strings last week?), but understanding this concept will make you a much stronger programmer and prevent mysterious bugs in your code.

Prerequisites

Before starting this week, you should be comfortable with:

- Creating and using strings (Week 4)
- String indexing and slicing (Week 4)
- Basic string methods like `upper()`, `lower()`, `replace()` (Week 4)
- Understanding variables and assignment (Module 1)

1.2 What You Already Know

You can work with strings using indexing, slicing, and methods. You know how to transform text using methods like `upper()` and `replace()`. You've seen that when you "change" a string, you often need to save the result. Now we'll discover WHY this happens and what's really going on behind the scenes!

1.3 What You'll Be Able to Do

By the end of this week, you'll:

- Understand what immutability means and why Python strings are immutable
- Draw memory diagrams showing how strings are stored and "modified"
- Predict the behavior of string operations using mental models
- Distinguish between creating new strings and attempting to modify existing ones
- Write efficient code that works with string immutability instead of against it
- Debug common string-related errors with confidence

2 VIDEO 1: Understanding String Immutability

2.1 Understanding Immutability

Immutability means "cannot be changed." When we say strings are immutable in Python, we mean that once a string is created, its contents can NEVER be modified. This might seem confusing - haven't we been "changing" strings all along? Let's uncover the truth!

2.2 The Immutability Revelation

```

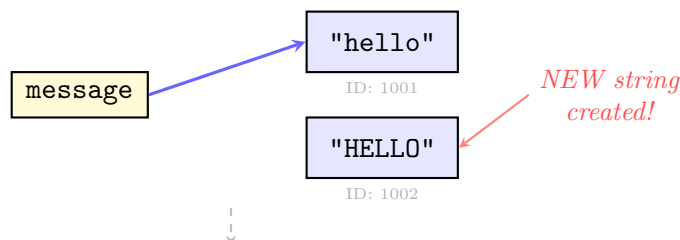
1 # This looks like we're changing the string...
2 message = "hello"
3 message = message.upper()
4 print(message) # HELLO
5
6 # But we're NOT! We created a NEW string!
7 # The original "hello" still exists (briefly)
8 # Python created "HELLO" and pointed message to it
    
```

2.3 Visualizing What Really Happens

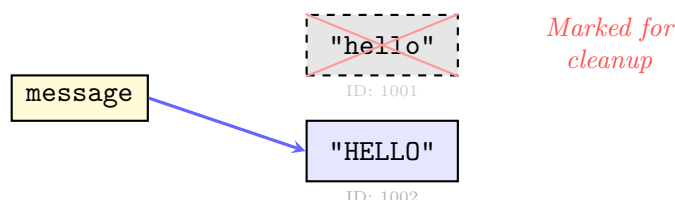
Stage 1: Initial String



Stage 2: After .upper() - TWO strings exist!



Stage 3: After assignment - Variable points to new string



The original "hello" was NEVER changed - we created a completely new string!

2.4 Proof of Immutability

```

1 # Try to change a character directly - ERROR!
2 word = "python"
    
```

```

3 # word[0] = "P" # This would cause: TypeError!
4
5 # Why? Because strings CANNOT be modified!
6 # Instead, we must create a new string:
7 word = "P" + word[1:] # Creates NEW string "Python"
8 print(word) # Python

```

2.5 Why Immutability Matters

Here's what happens when multiple variables reference the same string:

```

1 # Both variables point to the SAME string
2 original = "hello"
3 copy = original
4
5 # This creates a NEW string, doesn't change "hello"
6 copy = copy.upper()
7
8 print(original) # Still "hello" - unchanged!
9 print(copy)    # "HELLO" - new string

```

⚠ Common Error

Common Mistake: Thinking string methods modify the original string

```

1 text = "hello world"
2 text.upper() # This does nothing visible!
3 print(text)  # Still "hello world"
4
5 # Fix: Save the result!
6 text = text.upper()
7 print(text)   # "HELLO WORLD"

```

String methods return NEW strings - always save the result!

💡 Rule

The Golden Rule of Strings: You can never change a string. You can only create new strings. Every string operation returns a NEW string object!

👉 Quick Check

What will this print? Think about immutability:

```

1 name = "Alice"
2 friend = name
3 friend = friend.lower()
4 print(name)
5 print(friend)

```

Why doesn't changing friend affect name?

3 VIDEO 2: Memory Diagrams for Strings

3.1 Understanding String Storage

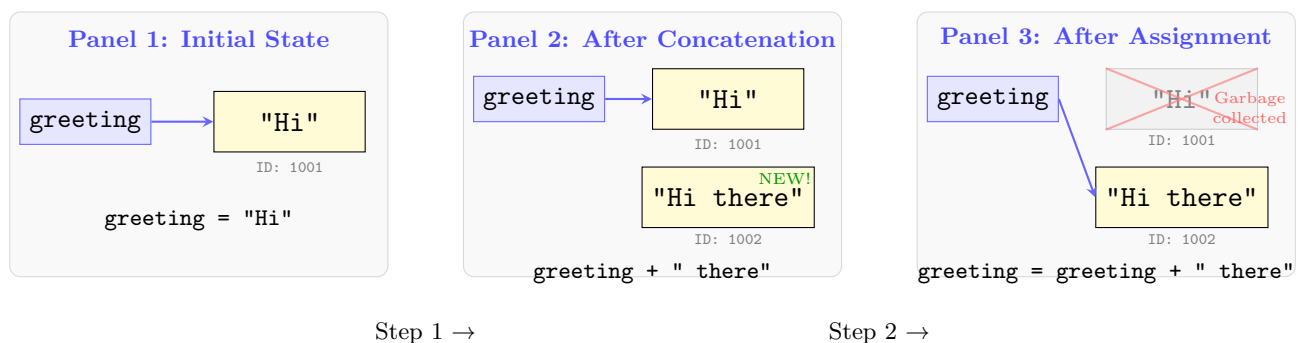
To truly understand immutability, we need to visualize how Python stores strings in memory. Memory diagrams are like X-ray vision for your code - they show you exactly what's happening behind the scenes!

3.2 Drawing Your First Memory Diagram

```

1 # Step 1: Create a string
2 greeting = "Hi"
3
4 # Step 2: Try to extend it
5 greeting = greeting + " there"
6
7 # What happened in memory?
8 # Two separate strings were created!
```

3.3 Memory Evolution Step by Step

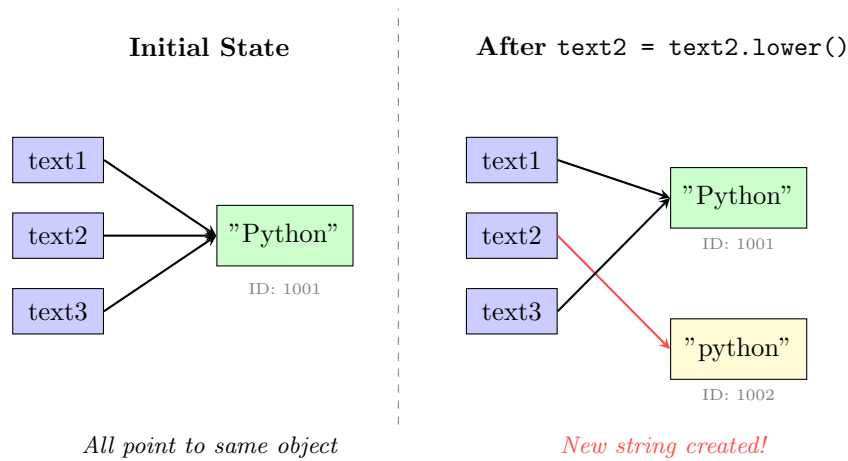


3.4 Multiple Variables, Same String

In this example we see how multiple variables can point to the same string object, and what happens when we “modify” one of them:

```

1 # Multiple variables can point to the same string
2 text1 = "Python"
3 text2 = text1 # Points to SAME string object
4 text3 = "Python" # Python optimizes - same object!
5
6 # But when we "modify"...
7 text2 = text2.lower() # Creates NEW string
8 # text1 is unchanged, text2 points to new "python"
```



Note: Python optimizes by reusing string objects when possible

3.5 String Operations Create New Objects

```

1 # Every operation creates a new string
2 word = "cat"           # Create string 1
3 word = word + "s"      # Create string 2: "cats"
4 word = word.upper()    # Create string 3: "CATS"
5 word = word[0:3]       # Create string 4: "CAT"
6
7 # Four different string objects were created!
8 # Previous versions are discarded

```

3.6 Memory Efficient Patterns

Since every operation creates a new string, we should minimize unnecessary operations:

```

1 # INEFFICIENT: Creates many intermediate strings
2 result = ""
3 for letter in "hello":
4     result = result + letter.upper() # 5 new strings!
5
6 # BETTER: Use join for multiple concatenations
7 letters = [letter.upper() for letter in "hello"]
8 result = "".join(letters) # Only 1 final string

```

i Tip

Memory Diagram Tips:

- Draw boxes for string values
- Draw arrows from variables to their strings
- Show new boxes for new strings (never modify existing boxes)
- Cross out abandoned strings (no arrows pointing to them)

Quick Check

Draw a memory diagram for this code:

```
1 a = "dog"
2 b = a
3 a = a + "s"
```

How many string objects exist after line 3? Which variables point where?

4 VIDEO 3: Creating New Strings vs Modifying

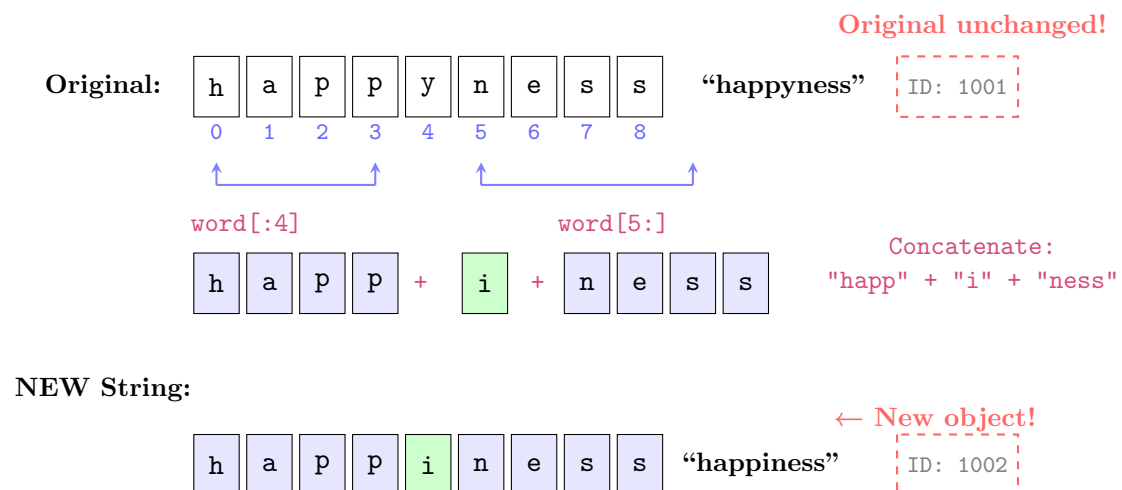
4.1 Understanding the Difference

Since strings are immutable, we can't modify them - but we CAN create new strings based on existing ones. Let's master the patterns for working WITH immutability instead of fighting against it!

4.2 Pattern 1: Character Replacement

```
1 # WRONG: Try to modify character
2 word = "happyness"
3 # word[4] = "i" # TypeError! Can't modify
4
5 # RIGHT: Create new string
6 word = word[:4] + "i" + word[5:]
7 print(word) # "happiness"
```

4.3 Visualization of String Building



4.4 Common String Creation Patterns

Here are the most common patterns for creating modified versions of strings:

```
1 original = "Hello World"
2
3 # Pattern 1: Replace all occurrences
4 new_text = original.replace("o", "0") # "Hell0 W0rld"
5
```

```

6 # Pattern 2: Change case
7 shouty = original.upper() # "HELLO WORLD"
8
9 # Pattern 3: Build from pieces
10 first = original[:5] # "Hello"
11 second = original[6:] # "World"
12 reversed_order = second + " " + first # "World Hello"

```

4.5 Building Strings Efficiently

When building strings from many pieces, collect the pieces first:

```

1 # Building a formatted message
2 parts = []
3 parts.append("Name: ")
4 parts.append("Alice")
5 parts.append(", Age: ")
6 parts.append("13")
7
8 # Join all at once - efficient!
9 message = "".join(parts)
10 print(message) # "Name: Alice, Age: 13"

```

4.6 Common Modification Scenarios

Task	Wrong Approach	Right Approach
Change one character	<code>s[i] = 'x'</code>	<code>s = s[:i] + 'x' + s[i+1:]</code>
Remove character	Modify in place	<code>s = s[:i] + s[i+1:]</code>
Insert character	Modify in place	<code>s = s[:i] + 'x' + s[i:]</code>
Append to string	<code>s[len(s)] = 'x'</code>	<code>s = s + 'x'</code>

⚠ Common Error

Common Mistake: Forgetting to save string method results

```

1 # WRONG - result is lost!
2 text = "  spaces  "
3 text.strip() # Returns new string, but we don't save it
4 print(text) # Still "  spaces  "
5
6 # RIGHT - save the result
7 text = text.strip()
8 print(text) # "spaces"

```

💡 Tip

String Reconstruction Pattern: `new_string = string[:index] + new_part + string[index+len(old_part):]`

Use slicing to extract parts, then concatenate to build new string.

👉 Quick Check

How would you change the 3rd character of "python" to 'T'? Write the code using slicing and concatenation.

5 VIDEO 4: Summary

5.1 What We Learned This Week

You've discovered one of Python's fundamental truths: strings are immutable! This isn't a limitation - it's a feature that makes strings predictable and safe to use. You now understand that every string operation creates a new string, you can visualize this with memory diagrams, and you know how to work efficiently with immutable strings.

5.2 The Power of Understanding Immutability

Understanding immutability helps you:

- **Predict behavior:** You know why `text.upper()` doesn't change `text`
- **Debug effectively:** You understand why certain operations "don't work"
- **Write efficient code:** You know when you're creating new strings
- **Avoid confusion:** You understand why changing one variable doesn't affect another
- **Think like a programmer:** You see beyond the surface to what's really happening

5.3 Key Insights We Discovered

What Seems to Happen	What Really Happens
String is modified	New string is created
<code>text.upper()</code> changes <code>text</code>	Returns new string (must save it!)
<code>s[0] = 'X'</code> changes first character	<code>TypeError</code> - cannot modify
<code>s = s + 'abc'</code> extends string	Creates entirely new string
Two variables share changes	Each variable independent after "change"

★ Landmark Moment

You now understand why strings can't change - they're immutable! This deep understanding separates beginners from real programmers. You can visualize what happens in memory, predict string behavior perfectly, and write code that works WITH immutability. This knowledge will help you understand lists (mutable!) next week!

6 Quick Reference

Quick Reference

Week 5 Essential Patterns:

String Immutability:

- Pattern: Strings cannot be modified after creation
- Common use: Understanding why operations return new strings
- Common mistake: Forgetting to save method results

Memory Diagrams:

- Pattern: Draw boxes for values, arrows for variables
- Memory: Each operation creates new string object
- Debug tip: Visualize to predict behavior

String Reconstruction:

- Pattern: `new = old[:i] + replacement + old[j:]`
- Common use: Simulating character changes
- Common mistake: Trying to use `string[i] = char`

Efficient String Building:

- Pattern: Collect pieces, then `"".join(pieces)`
- Memory: Avoids creating many intermediate strings
- Debug tip: Count how many strings your code creates

6.1 Reflection Questions

1. Why do you think Python designers made strings immutable? What problems does this prevent?
2. Draw a memory diagram for: `a = "hi"; b = a; b = b + "!"`. How many strings exist at the end?
3. You tried to change a string character and got a `TypeError`. Explain to a friend why this happened and how to fix it.
4. When you type `text.upper()` and nothing seems to happen, what's really going on? Why isn't `text` changing?
5. Imagine strings WERE mutable (changeable). What confusing situations might occur when two variables point to the same string?

Next week: We'll explore advanced string operations - splitting, joining, searching, and formatting. Your deep understanding of immutability will make these powerful tools easy to master!